

PRIMERA EVALUACIÓN

Asignatura	Tecnologías Emergentes I	Unidad(es)	Unidad 1: Introducción a Cloud Computing Unidad 2: Proveedores de Cloud
Realización	Miércoles, 30 de septiembre de 2026	Equipo e integrantes	Aldira Mamani Sanga Brenda Michelle Alanoca Challco

OBJETIVO

Integrar conocimientos de Cloud Computing y habilidades de Docker, Kubernetes y AWS mediante el diseño e implementación de un sistema de registro y consulta de productos.

LINEAMIENTOS

- La entrega debe realizarse hasta el miércoles 30 de septiembre a las 08:00 am.
- Se deben organizar equipos de hasta 3 estudiantes. No se aceptarán trabajos individuales.
- Todos los integrantes del equipo deberán participar activamente en el desarrollo.
- Cada uno de los integrantes del equipo debe subir y entregar su documento de proyecto.
- Todas las respuestas deberán ser comprendidas y defendidas técnicamente durante las validaciones realizadas por el docente.

SITUACIÓN PROBLEMÁTICA

Lomax SA comercializa productos tecnológicos de diferentes categorías. Actualmente, el personal registra nombres, códigos y precios en hojas de cálculo, mientras las fotografías se comparten por mensajería y se guardan en carpetas de distintas computadoras. Cuando un cliente solicita información, el vendedor debe buscar los datos y localizar manualmente la imagen correspondiente.

Esta forma de trabajo provoca códigos duplicados, productos sin fotografía y diferencias entre las listas que utilizan los vendedores. Además, las características no son iguales para todos los artículos, por ejemplo: un teclado requiere indicar su conexión y distribución; una pantalla, sus pulgadas y resolución. Las hojas existentes no permiten organizar estos atributos de manera consistente.

Las fotografías de gran tamaño hacen lenta la consulta y, cuando se reemplaza una computadora o falla la aplicación, el equipo tiene dificultades para recuperar la información. Con la incorporación de nuevos vendedores, la empresa necesita que varias personas consulten el mismo catálogo y que un fallo de la aplicación no obligue a registrar nuevamente los productos.

La empresa requiere una plataforma que permita registrar cada producto una sola vez, asociar sus características y fotografía, y mostrarlo en un catálogo común cuando su información esté completa. El sistema deberá rechazar datos inválidos, generar imágenes pequeñas para facilitar la consulta y permitir reintentar registros incompletos sin duplicarlos.

El alcance se limita a registrar productos, mostrar el catálogo y consultar el detalle de un producto. La solución debe ser desarrollada en **FLOCI**.

ARQUITECTURA DE LA SOLUCIÓN

Frontend y reverse proxy

- El frontend tendrá formulario de registro, catálogo con tarjetas y detalle del producto. Mostrará nombre, precio, categoría, fotografía y atributos.
- El reverse proxy será la entrada de usuarios y dirigirá las solicitudes al frontend o a la API.

RDS y DynamoDB

- RDS almacenará categorías y productos: producto_id, código único, nombre, descripción, precio decimal, categoria_id, fecha y estado PENDIENTE o PUBLICADO. Exigirá campos obligatorios, precio no negativo y categoría existente.
- DynamoDB almacenará atributos variables y datos de imagen por producto_id: imagen_original_key, miniatura_key y estado de procesamiento. El identificador será el mismo de RDS; la relación entre bases se verificará desde la API, no mediante una clave foránea entre servicios.

S3 y Lambda

- S3 utilizará dos buckets: uno para las fotografías **originales** y otro para las **miniaturas** generadas por Lambda.
- Después de guardar el original, la API invocará Lambda de forma síncrona. La función validará la imagen, generará una miniatura de hasta 300 × 300 píxeles, conservando la proporción, y actualizará en DynamoDB su clave y estado LISTA o ERROR.

ECR y EKS

- ECR almacenará las imágenes Docker propias del frontend y backend.
- EKS ejecutará los componentes de la aplicación con esas imágenes y permitirá verificar escalamiento y autorrecuperación.

Flujo integral y publicación

- La API registra el producto en RDS como PENDIENTE, guarda sus atributos en DynamoDB, carga la fotografía en S3 e invoca Lambda.
- Solo cambia a PUBLICADO cuando confirma atributos y miniatura disponibles.
- El catálogo combina datos de RDS y DynamoDB y recupera las imágenes de S3 a través de la API.

SABER CONOCER

Actividad 1. Preparación conceptual

Investigar y sintetizar los conceptos necesarios para justificar la arquitectura y las decisiones técnicas del proyecto.

La preparación deberá abordar:

Cloud Computing

Concepto	Definición
Características del Cloud Computing	El Cloud Computing es un modelo que permite utilizar recursos informáticos, como servidores, almacenamiento, bases de datos y aplicaciones, a través de Internet. Sus principales características son el acceso bajo demanda, escalabilidad, disponibilidad, administración centralizada y pago según el uso en muchos servicios. En el proyecto Lomax permite utilizar servicios de AWS mediante FLOCI sin depender de una sola computadora.
Escalabilidad vertical y horizontal	La escalabilidad vertical consiste en aumentar los recursos de una instancia, por ejemplo, más CPU o memoria. La escalabilidad horizontal consiste en agregar más instancias o réplicas de una aplicación. En el proyecto, EKS permite trabajar con varias réplicas del backend para atender más solicitudes.
Virtualización	Es la tecnología que permite crear recursos virtuales, como máquinas, redes o servidores, utilizando recursos físicos compartidos. Permite aprovechar mejor la infraestructura y ejecutar diferentes servicios de manera aislada.
IaaS, PaaS y SaaS	IaaS proporciona infraestructura virtual como servidores, redes y almacenamiento. PaaS proporciona una plataforma administrada para desarrollar y ejecutar aplicaciones. SaaS proporciona aplicaciones listas para utilizar. AWS ofrece servicios que pueden utilizarse bajo diferentes niveles de administración según la solución.
Tipos de nube	Las principales modalidades son nube pública , donde los recursos son proporcionados por un proveedor externo; nube privada , utilizada exclusivamente por una organización; nube híbrida , que combina nube pública y privada; y multinube , que utiliza servicios de diferentes proveedores cloud.
Ventajas y limitaciones de los servicios cloud	Entre las ventajas están la escalabilidad, disponibilidad, acceso remoto, recuperación ante fallos y reducción de la necesidad de administrar infraestructura física. Entre sus limitaciones se encuentran la dependencia de

	Internet, costos variables, configuración incorrecta de servicios y dependencia del proveedor.
--	--

Contenedores

Concepto	Definición
Concepto de contenedor	Un contenedor es una unidad de software que incluye una aplicación y los elementos necesarios para ejecutarla. Permite que la aplicación funcione de manera consistente en diferentes entornos.
Imagen y contenedor	Una imagen es una plantilla inmutable que contiene el código, dependencias y configuración necesarios para ejecutar una aplicación. Un contenedor es una instancia en ejecución de esa imagen. En Lomax se crearán imágenes Docker del frontend y backend para almacenarlas en ECR.
Dockerfile	Es un archivo de instrucciones utilizado para construir una imagen Docker. Permite especificar la imagen base, instalar dependencias, copiar archivos y definir el comando que ejecutará la aplicación.
Redes	Las redes Docker permiten que diferentes contenedores se comuniquen entre sí de manera controlada. Facilitan la conexión entre componentes como frontend, backend y otros servicios.
Volúmenes	Los volúmenes permiten almacenar información fuera del sistema de archivos temporal del contenedor. Esto permite conservar datos aunque el contenedor sea eliminado o reemplazado.
Docker Compose	Docker Compose es una herramienta que permite definir y ejecutar múltiples contenedores mediante un archivo de configuración, normalmente docker-compose.yml. Facilita la ejecución conjunta de servicios relacionados.
Ventajas y limitaciones de Docker	Docker facilita la portabilidad, aislamiento, distribución y despliegue de aplicaciones. También permite construir entornos reproducibles. Como limitaciones, requiere conocimientos de redes, almacenamiento y seguridad, y los contenedores no sustituyen por sí mismos mecanismos completos de orquestación.

Kubernetes

Concepto	Definición
----------	------------

Arquitectura básica	Kubernetes utiliza una arquitectura de tipo control plane + nodos de trabajo . El control plane administra el estado del clúster, mientras los nodos ejecutan las cargas de trabajo mediante Pods.
Cluster	Un clúster es el conjunto de recursos que Kubernetes administra como una unidad. Está compuesto por el plano de control y los nodos donde se ejecutan las aplicaciones.
Pod	Es la unidad mínima de ejecución de Kubernetes. Un Pod contiene uno o más contenedores que comparten recursos como red y almacenamiento. En Lomax los componentes frontend y backend pueden ejecutarse mediante Pods.
Deployment	Es un recurso de Kubernetes utilizado para administrar réplicas de Pods. Permite realizar despliegues, actualizaciones y mantener disponible la cantidad deseada de instancias de una aplicación.
Service	Es un recurso que proporciona una forma estable de acceder a un conjunto de Pods. Permite que otros componentes o usuarios accedan a una aplicación sin depender directamente de la dirección IP de un Pod específico.
Escalamiento	Es el proceso de aumentar o disminuir la cantidad de réplicas de una aplicación según las necesidades de carga. Kubernetes permite realizar escalamiento manual y, mediante mecanismos adicionales, automático.
Autorecuperación	Kubernetes puede detectar que un Pod dejó de funcionar y crear otro para mantener el estado deseado definido por el Deployment. Esto permite mejorar la disponibilidad de las aplicaciones.

AWS

Concepto	Definición
RDS	Amazon Relational Database Service (RDS) es un servicio administrado para bases de datos relacionales. Facilita tareas como configuración, operación y mantenimiento de motores de bases de datos. En Lomax almacenará las categorías y productos, incluyendo código, nombre, descripción, precio, categoría, fecha y estado.
DynamoDB	Amazon DynamoDB es una base de datos NoSQL administrada que utiliza un modelo basado en tablas y claves. Es adecuada para almacenar información con estructuras flexibles. En Lomax se utilizará para almacenar los atributos variables de los productos y los datos relacionados con el procesamiento de imágenes.

S3	Amazon Simple Storage Service (S3) es un servicio de almacenamiento de objetos. Permite almacenar archivos mediante buckets. En Lomax se utilizarán dos buckets: uno para fotografías originales y otro para las miniaturas generadas.
Lambda	AWS Lambda es un servicio de ejecución de código sin necesidad de administrar servidores directamente. La función puede ejecutarse como respuesta a una invocación. En Lomax se utilizará para validar las fotografías, generar miniaturas de hasta 300 × 300 píxeles y actualizar el estado de procesamiento en DynamoDB.
ECR	Amazon Elastic Container Registry (ECR) es un registro administrado para almacenar y distribuir imágenes de contenedores. En Lomax almacenará las imágenes Docker del frontend y backend que posteriormente serán utilizadas por EKS.
EKS	Amazon Elastic Kubernetes Service (EKS) es un servicio administrado para ejecutar Kubernetes en AWS. Permite desplegar y administrar aplicaciones contenedorizadas utilizando Pods, Deployments y Services. En Lomax será utilizado para ejecutar los componentes de frontend y backend y comprobar características como escalamiento y autorecuperación.

Resultado

Producto 1: Síntesis conceptual.

Entregar las tablas completas y conocer la relación de los conceptos con la solución.

SABER HACER

Realizar las etapas siguientes en orden. Preparar al menos 20 productos publicados, distribuidos en 3 categorías, con dos conjuntos distintos de atributos y una fotografía por producto.

Etapla 1. Diseño de la arquitectura

Diseñar el registro y la consulta del catálogo con **íconos oficiales de AWS** para los servicios. Señalar expresamente el entorno local FLOCI, frontend, proxy, API, redes, protocolos, puertos, almacenamiento y ubicación de los componentes en Kubernetes.

Entregable P2: diagrama de arquitectura.

Verificación E1: explicar el funcionamiento de la solución utilizando el diagrama. Al finalizar, contrastar el diagrama con los recursos desplegados.



Etapa 2. Persistencia con RDS y DynamoDB

Crear la base RDS y la tabla DynamoDB. Entregar un modelo relacional, esquema de atributos y scripts reproducibles de creación y carga.

- RDS: probar inserción válida, código duplicado, precio negativo y categoría inexistente. Las operaciones inválidas deben rechazarse sin dejar registros parciales.
- DynamoDB: guardar y recuperar por producto_id, productos con atributos diferentes. Verificar que cada identificador corresponda al producto esperado en RDS. Las referencias de imágenes podrán estar pendientes hasta la etapa 3.

Entregable P3: RDS y DynamoDB creados y con datos cargados.

Verificación E2: consultas SQL y get-item de DynamoDB, salidas de las pruebas de restricciones y comparación de identificadores y valores antes y después de reiniciar los componentes de persistencia sin borrar volúmenes. La carga inicial debe poder repetirse sin duplicar registros.

E1-01. Verificación del entorno FLOCI

Se inicia el entorno local FLOCI mediante Docker y se verifica que el emulador se encuentra activo y

disponible mediante el endpoint `http://localhost:4566`. La respuesta corresponde al servicio S3 de AWS y confirma la comunicación con FLOCI.

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
04557b2c3671   mysql:8.0     "docker-entrypoint.s..." 29 hours ago  Up About an hour  0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
c580b34931d1   floci/floci:latest "/usr/local/bin/dock..." 4 days ago    Up 17 minutes (healthy)  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:4566->4566/tcp, [::]:4566->4566/tcp
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> curl.exe http://localhost:4566
<?xml version="1.0" encoding="UTF-8"?><ListAllMyBucketsResult xmlns="http://s3.amazonaws.com/doc/2006-03-01/"><Owner><ID>owner</ID><DisplayName>owner</DisplayName></Owner><Buckets></Buckets></ListAllMyBucketsResult>
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
04557b2c3671   mysql:8.0     "docker-entrypoint.s..." 29 hours ago  Up About an hour  0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
c580b34931d1   floci/floci:latest "/usr/local/bin/dock..." 4 days ago    Up 17 minutes (healthy)  0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:4566->4566/tcp, [::]:4566->4566/tcp
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws sts get-caller-identity --endpoint-url=http://localhost:4566
{
  "UserId": "000000000000",
  "Account": "000000000000",
  "Arn": "arn:aws:iam::000000000000:root"
}
```

E2-01 — Instancia RDS Lomax creada

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws rds describe-db-instances `
>> --db-instance-identifier lomax-rds `
>> --endpoint-url=http://localhost:4566 `
>> --query "DBInstances[0].DBInstanceStatus"
"available"
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

E2-01 — Creación y disponibilidad de RDS MySQL en FLOCI

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws rds describe-db-instances `
>> --db-instance-identifier lomax-rds `
>> --endpoint-url=http://localhost:4566
{
  "DBInstances": [
    {
      "DBInstanceIdentifier": "lomax-rds",
      "DBInstanceClass": "db.t3.micro",
      "Engine": "mysql",
      "DBInstanceStatus": "available",
      "MasterUsername": "admin",
      "Endpoint": {
        "Address": "172.19.0.2",
        "Port": 7001
      },
      "AllocatedStorage": 20,
    }
  ]
}
```

E2-02 — Conexión al motor MySQL de RDS mediante FLOCI


```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> docker run --rm -it --network floci-red mysql:8.0 mysql -h 172.19.0.2 -P 7001 -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> 
```

E2-03 — Creación de la base de datos Lomax

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
mysql> USE lomax;
Database changed
mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| admin    |
| information_schema |
| lomax    |
| mysql    |
| performance_schema |
| sys      |
+-----+
6 rows in set (0.03 sec)

mysql> 
```

E2-04 — Tablas principales de la base de datos Lomax

```
mysql> SHOW TABLES;
+-----+
| Tables_in_lomax |
+-----+
| categorias       |
| productos        |
+-----+
2 rows in set (0.00 sec)

mysql> 
```

E2-04 — Estructura de tablas y restricciones de RDS

```
mysql> DESCRIBE productos;
```

Field	Type	Null	Key	Default	Extra
producto_id	int	NO	PRI	NULL	auto_increment
codigo	varchar(50)	NO	UNI	NULL	
nombre	varchar(150)	NO		NULL	
descripcion	text	YES		NULL	
precio	decimal(10,2)	NO		NULL	
categoria_id	int	NO	MUL	NULL	
fecha_creacion	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
estado	enum('PENDIENTE','PUBLICADO')	NO		PENDIENTE	

```
8 rows in set (0.00 sec)
```

E2-05 — Carga inicial de categorías

```
mysql> SELECT * FROM categorias;
```

categoria_id	nombre	descripcion
1	Laptops	Computadoras porttiles
2	Monitores	Monitores para computadora
3	Perifricos	Teclados, mouse y accesorios
4	Celulares	Telfonos inteligentes
5	Componentes	Componentes para computadoras

```
5 rows in set (0.00 sec)
```

E2-06 — Inserción válida de producto

```
mysql> SELECT * FROM productos;
```

producto_id	codigo	nombre	descripcion	precio	categoria_id	fecha_creacion	estado
1	LOM-001	Laptop Lenovo IdeaPad	Laptop para uso acadmico y profesional	4500.00	1	2026-09-30 01:56:13	PENDIENTE

```
1 row in set (0.01 sec)

mysql>
```

E2-07 — Rechazo de código de producto duplicado sin registro parcial

```
mysql> INSERT INTO productos
-> (codigo, nombre, descripcion, precio, categoria_id, estado)
-> VALUES
-> ('LOM-001',
-> 'Laptop HP Pavilion',
-> 'Segundo producto con cdigo repetido',
-> 5200.00,
-> 1,
-> 'PENDIENTE');
```

```
ERROR 1062 (23000): Duplicate entry 'LOM-001' for key 'productos.codigo'
```

```
mysql> SELECT * FROM productos;
```

producto_id	codigo	nombre	descripcion	precio	categoria_id	fecha_creacion	estado
1	LOM-001	Laptop Lenovo IdeaPad	Laptop para uso acadmico y profesional	4500.00	1	2026-09-30 01:56:13	PENDIENTE

```
1 row in set (0.00 sec)
```

E2-08 — Rechazo de precio negativo sin registro parcial

```
mysql> INSERT INTO productos
-> (codigo, nombre, descripcion, precio, categoria_id, estado)
-> VALUES
-> ('LOM-002',
-> 'Monitor Samsung',
-> 'Monitor de prueba con precio negativo',
-> -1500.00,
-> 2,
-> 'PENDIENTE');
ERROR 3819 (HY000): Check constraint 'chk_precio_no_negativo' is violated.
mysql> SELECT * FROM productos;
```

producto_id	codigo	nombre	descripcion	precio	categoria_id	fecha_creacion	estado
1	LOM-001	Laptop Lenovo IdeaPad	Laptop para uso academico y profesional	4500.00	1	2026-09-30 01:56:13	PENDIENTE

1 row in set (0.26 sec)

E2-09 — Rechazo de categoría inexistente sin registro parcial

```
mysql> INSERT INTO productos
-> (codigo, nombre, descripcion, precio, categoria_id, estado)
-> VALUES
-> ('LOM-003',
-> 'Teclado Logitech',
-> 'Producto con categoria inexistente',
-> 350.00,
-> 999,
-> 'PENDIENTE');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`lomax`.`productos`, CONSTRAINT `fk_producto_categoria` FOREIGN KEY (`categoria_id`) REFERENCES `categorias` (`categoria_id`))
mysql> SELECT * FROM productos;
```

producto_id	codigo	nombre	descripcion	precio	categoria_id	fecha_creacion	estado
1	LOM-001	Laptop Lenovo IdeaPad	Laptop para uso academico y profesional	4500.00	1	2026-09-30 01:56:13	PENDIENTE

1 row in set (0.00 sec)

E2-10 — Creación de tabla ProductoAtributos en DynamoDB

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws dynamodb describe-table `
>> --table-name ProductoAtributos `
>> --endpoint-url=http://localhost:4566
{
  "Table": {
    "AttributeDefinitions": [
      {
        "AttributeName": "producto_id",
        "AttributeType": "N"
      }
    ],
    "TableName": "ProductoAtributos",
    "KeySchema": [
```

E2-11 — Consulta de atributos variables del producto en DynamoDB

```

PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> aws dynamodb get-item `
>> --table-name ProductoAtributos `
>> --key file:///database/clave-producto-1.json `
>> --endpoint-url=http://localhost:4566
    "S": "16GB"
  },
  "almacenamiento": {
    "S": "512GB SSD"
  }
},
"imagen_original_key": {
  "S": ""
},
"miniatura_key": {
  "S": ""
},
"estado_procesamiento": {
  "S": "PENDIENTE"
}

```

E2-12 - DynamoDB sin duplicados (Count: 1)

```

PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> aws dynamodb scan `
>> --table-name ProductoAtributos `
>> --endpoint-url=http://localhost:4566
    "S": ""
  },
  "miniatura_key": {
    "S": ""
  },
  "estado_procesamiento": {
    "S": "PENDIENTE"
  }
},
"Count": 1,
"ScannedCount": 1,
"ConsumedCapacity": null
}

```

E2-13 — Correspondencia de producto entre RDS y DynamoDB.

```

PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> docker run --rm -it --network floci-red mysql:8.0 mysql -h 172.19.0.2 -P 7001 -u admin -p
Database changed
mysql>
mysql> SELECT
  -> producto_id,
  -> codigo,
  -> nombre,
  -> precio,
  -> categoria_id,
  -> estado
  -> FROM productos;
+-----+-----+-----+-----+-----+-----+
| producto_id | codigo | nombre                | precio | categoria_id | estado |
+-----+-----+-----+-----+-----+-----+
|          1 | LOM-001 | Laptop Lenovo IdeaPad | 4500.00 |          1 | PENDIENTE |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql>

```

Etapa 3. AWS con S3 y Lambda

Crear los buckets. Aceptar JPEG o PNG válidos de hasta 5 Mb. Lambda recibe producto_id, bucket y clave del original; produce una miniatura proporcional de máximo 300 × 300 píxeles y guarda su referencia y estado en DynamoDB.

- Invocar la función mediante CLI o script durante esta etapa; el backend asumirá luego la invocación síncrona. Usar una clave de salida determinista por imagen para que repetir la solicitud no cree miniaturas adicionales.
- No publicar el producto cuando la imagen sea inválida o falle el procesamiento. Conservar el estado y permitir reintentar.

E3-01: Captura de los dos buckets creados.

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws s3api list-buckets
>> --endpoint-url=http://localhost:4566
{
  "Name": "lomax-originales",
  "CreationDate": "2026-09-30T03:00:59+00:00"
},
{
  "Name": "lomax-miniaturas",
  "CreationDate": "2026-09-30T03:01:09+00:00"
}
],
"Owner": {
  "DisplayName": "owner",
  "ID": "owner"
},
"Prefix": null
}
```

E3-02 — Crear la estructura de carpetas

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws s3api list-objects-v2
>> --bucket lomax-originales
>> --endpoint-url=http://localhost:4566
>> --bucket lomax-miniaturas
>> --endpoint-url=http://localhost:4566
{
  "Contents": [
    {
      "Key": "miniaturas/",
      "LastModified": "2026-09-30T03:03:20+00:00",
      "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
      "Size": 0,
      "StorageClass": "STANDARD"
    }
  ],
  "RequestCharged": null,
  "Prefix": ""
}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

E3-03 — Crear una imagen de prueba

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
es:',img.size); print('Formato:',img.format); p
Dimensiones: (1200, 800)
Formato: JPEG
Tamaño: 1200 x 800
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

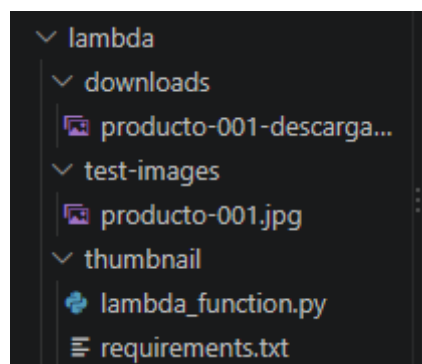
E3-04 — Subir la imagen original a S3

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws s3api list-objects-v2 `
>> --bucket lomax-originales `
>> --prefix originales/1/ `
>> --endpoint-url=http://localhost:4566
{
  "Contents": [
    {
      "Key": "originales/1/producto-001.jpg",
      "LastModified": "2026-09-30T04:02:54+00:00",
      "ETag": "\"9dc19f56385de838e5a216b85d4677b5\"",
      "Size": 27053,
      "StorageClass": "STANDARD"
    }
  ],
  "RequestCharged": null,
  "Prefix": "originales/1/"
}
```

E3-05 — Descargar y comprobar la imagen desde S3

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> python -c "from PIL imp
Dimensiones:',img.size); print('Formato:',img.format)"
Dimensiones: (1200, 800)
Formato: JPEG
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

E3-06 — Preparar la Lambda



E3-07 — Código de la Lambda

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Content .\lambda\thumbnail\lambda_function.py | Select-Object -First 10
import json
import os
import io

import boto3
from PIL import Image

s3 = boto3.client(
    "s3",
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3-08 — Instalar dependencias y crear ZIP

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Compress-Archive `
>> -Path .\lambda\thumbnail\build\* `
>> -DestinationPath .\lambda\thumbnail\lambda_function.zip `
>> -Force
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Item .\lambda\thumbnail\lambda_function.zip |
>> Select-Object Name, Length, FullName

Name                               Length FullName
----                               -
lambda_function.zip 25137966 C:\Users\WINDOWS 11\Desktop\lomax-catalogo\lambda\thumbnail\lambda_function.zip

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3-09 — Crear rol IAM para Lambda

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws iam create-role `
>> --role-name lomax-lambda-role `
>> --assume-role-policy-document file://lambda/thumbnail/trust-policy.json `
>> --endpoint-url=http://localhost:4566
{
  "Role": {
    "Path": "/",
    "RoleName": "lomax-lambda-role",
    "RoleId": "AROAWKC3MITTVGUKGIRX",
    "Arn": "arn:aws:iam::000000000000:role/lomax-lambda-role",
    "CreateDate": "2026-09-30T05:07:59.777417+00:00",
    "AssumeRolePolicyDocument": {
      "Version": "2012-10-17",
      "Statement": [
        {
          "Effect": "Allow",
          "Principal": {
            "Service": "lambda.amazonaws.com"

```

E3-10 — Función Lambda creada y configurada.

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda create-function `
>> --function-name lomax-thumbnail `
>> --runtime python3.10 `
>> --handler lambda_function.lambda_handler `
>> --role arn:aws:iam::000000000000:role/lomax-lambda-role ` ...
    "Size": 512
  },
  "SnapStart": {
    "ApplyOn": "None",
    "OptimizationStatus": "Off"
  },
  "RuntimeVersionConfig": {
    "RuntimeVersionArn": "arn:aws:lambda:us-east-1::runtime:5317a27786b53351485f427ffc031a740f27f00f1729ab5800e2be756037ed83"
  },
  "LoggingConfig": {
    "LogFormat": "Text",
    "LogGroup": "/aws/lambda/lomax-thumbnail"
  }
}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3-11 — Paquete Lambda compatible con Linux

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> tar -tf .\lambda\thumbnail\lambda_function.zip |
>> Select-String "x86_64-linux-gnu.so" |
>> Select-Object -First 10

PIL/_avif.cpython-310-x86_64-linux-gnu.so
PIL/_imaging.cpython-310-x86_64-linux-gnu.so
PIL/_imagingcms.cpython-310-x86_64-linux-gnu.so
PIL/_imagingft.cpython-310-x86_64-linux-gnu.so
PIL/_imagingmath.cpython-310-x86_64-linux-gnu.so
PIL/_imagingmorph.cpython-310-x86_64-linux-gnu.so
PIL/_imagingtk.cpython-310-x86_64-linux-gnu.so
PIL/_webp.cpython-310-x86_64-linux-gnu.so
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3-12 — Actualizar la Lambda

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda update-function-code `
>> --function-name lomax-thumbnail `
>> --zip-file fileb://lambda/thumbnail/lambda_function.zip `
>> --endpoint-url=http://localhost:4566
    "FunctionArn": "arn:aws:lambda:us-east-1:000000000000:function:lomax-thumbnail",
    "Runtime": "python3.10",
    "Role": "arn:aws:iam::000000000000:role/lomax-lambda-role",
    "Handler": "lambda_function.lambda_handler",
    "CodeSize": 25913366,
    "Timeout": 30,
    "MemorySize": 512,
    "Version": "1"
}

```



```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda update-function-code `
>> --function-name lomax-thumbnail `
>> --zip-file fileb://lambda/thumbnail/lambda_function.zip `
>> --endpoint-url=http://localhost:4566
    "AWS_REGION": "us-east-1",
    "ORIGINAL_BUCKET": "lomax-originales",
    "THUMBNAIL_BUCKET": "lomax-miniaturas",
    "DYNAMODB_TABLE": "ProductoAtributos"
  }
},
"TracingConfig": {
  "Mode": "PassThrough"
},
"RevisionId": "d943d4bf-f2d3-46bf-b66d-a198d586049e",
"State": "Active",
"LastUpdateStatus": "Successful",
"PackageType": "Zip"
}

```

E3-13 — Cuarto intento de la invocación

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda invoke `
>> --function-name lomax-thumbnail `
>> --payload fileb://lambda/thumbnail/evento-1.json `
>> --cli-binary-format raw-in-base64-out `
>> --endpoint-url=http://localhost:4566 `
>> .\lambda\thumbnail\respuesta-4.json
{
  "statusCode": 200,
  "ExecutedVersion": "$LATEST"
}

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Content .\lambda\thumbnail\respuesta-4.json
{"statusCode": 200, "producto_id": 1, "estado": "LISTA", "imagen_original_key": "originales/1/producto-001.jpg", "miniatura_key": "miniaturas/1/producto-001.jpg", "width": 300, "height": 200}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3-14 -- Miniatura almacenada en S3.

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws s3api list-objects-v2 `
>> --bucket lomax-miniaturas `
>> --prefix miniaturas/1/ `
>> --endpoint-url=http://localhost:4566
{
  "Contents": [
    {
      "Key": "miniaturas/1/producto-001.jpg",
      "LastModified": "2026-09-30T05:35:32+00:00",
      "ETag": "\"adf21173ae8292cd38161a87aa623d28\"",
      "Size": 3725,
      "StorageClass": "STANDARD"
    }
  ],
  "RequestCharged": null,
  "Prefix": "miniaturas/1/"
}

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3 — Verificación de miniatura 300×200

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws s3api get-object `
>> --bucket lomax-miniaturas `
>> --key miniaturas/1/producto-001.jpg `
>> .\lambda\downloads\miniatura-001.jpg `
>> --endpoint-url=http://localhost:4566
{
  "AcceptRanges": "bytes",
  "LastModified": "2026-09-30T05:35:32+00:00",
  "ContentLength": 3725,
  "ETag": "\"adf21173ae8292cd38161a87aa623d28\"",
  "ChecksumCRC32": "RmIZbw==",
  "ChecksumType": "FULL_OBJECT",
  "ContentType": "image/jpeg",
  "Metadata": {},
  "StorageClass": "STANDARD"
}

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> python -c "from PIL
s:',img.size); print('Formato:',img.format); print('Tamaño:',img.wi
Dimensiones: (300, 200)
Formato: JPEG

```

E3 — DynamoDB actualizado por Lambda

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws dynamodb get-item `
>> --table-name ProductoAtributos `
>> --key file:///database/clave-producto-1.json `
>> --endpoint-url=http://localhost:4566
{
  "S": "512GB SSD"
}
},
"imagen_original_key": {
  "S": "originales/1/producto-001.jpg"
},
"miniatura_key": {
  "S": "miniaturas/1/producto-001.jpg"
},
"estado_procesamiento": {
  "S": "LISTA"
}
}
}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>

```

E3 — Reprocesamiento sin duplicación de miniatura

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Content .\lambda\thumbnail\respuesta-reintento.json
{"Key": "miniaturas/1/producto-001.jpg", "width": 300, "height": 200}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws s3api list-objects-v2 `
>> --bucket lomax-miniaturas `
>> --prefix miniaturas/1/ `
>> --endpoint-url=http://localhost:4566
{
  "Contents": [
    {
      "Key": "miniaturas/1/producto-001.jpg",
      "LastModified": "2026-09-30T05:43:15+00:00",
      "ETag": "\"adf21173ae8292cd38161a87aa623d28\"",
      "Size": 3725,
      "StorageClass": "STANDARD"
    }
  ],
  "RequestCharged": null,
  "Prefix": "miniaturas/1/"
}

```

E3 — Rechazo de imagen inválida

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda invoke `
>> --function-name lomax-thumbnail `
>> --payload fileb://lambda/thumbnail/evento-invalido.json `
>> --cli-binary-format raw-in-base64-out `
>> --endpoint-url=http://localhost:4566 `
>> .\lambda\thumbnail\respuesta-invalida.json
{
  "statusCode": 200,
  "ExecutedVersion": "$LATEST"
}

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Content .\lambda\thumbnail\respuesta-invalida.json
{"statusCode": 400, "producto_id": 1, "estado": "ERROR", "error": "cannot identify image file <_io.BytesIO object at 0x7f55ad32f2e0>"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

E3 — Estado ERROR registrado en DynamoDB

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws dynamodb get-item `
>> --table-name ProductoAtributos `
>> --key file://database/clave-producto-1.json `
>> --endpoint-url=http://localhost:4566
      "S": "512GB SSD"
    }
  },
  "imagen_original_key": {
    "S": "originales/1/imagen-invalida.jpg"
  },
  "miniatura_key": {
    "S": "miniaturas/1/producto-001.jpg"
  },
  "estado_procesamiento": {
    "S": "ERROR"
  }
}
```

E3 — Restauración exitosa después de error

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda invoke `
>> --function-name lomax-thumbnail `
>> --payload fileb://lambda/thumbnail/evento-1.json `
>> --cli-binary-format raw-in-base64-out `
>> --endpoint-url=http://localhost:4566 `
>> .\lambda\thumbnail\respuesta-restauracion.json
{
  "statusCode": 200,
  "ExecutedVersion": "$LATEST"
}

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Content .\lambda\thumbnail\respuesta-restauracion.json
{"statusCode": 200, "producto_id": 1, "estado": "LISTA", "imagen_original_key": "originales/1/producto-001.jpg", "miniatura_key": "miniaturas/1/producto-001.jpg", "width": 300, "height": 200}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

E3 — Lambda rechaza imagen superior a 5 MB

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> aws lambda invoke `
>> --function-name lomax-thumbnail `
>> --payload fileb://lambda/thumbnail/evento-grande.json `
>> --cli-binary-format raw-in-base64-out `
>> --endpoint-url=http://localhost:4566 `
>> .\lambda\thumbnail\respuesta-grande.json
{
  "statusCode": 200,
  "ExecutedVersion": "$LATEST"
}

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo> Get-Content .\lambda\thumbnail\respuesta-grande.json
{"statusCode": 400, "producto_id": 1, "estado": "ERROR", "error": "La imagen supera el limite de 5 MB"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo>
```

E3 — DynamoDB registra ERROR para imagen superior a 5 MB

```
PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> aws dynamodb get-item `
>> --table-name ProductoAtributos `
>> --key file://database/clave-producto-1.json `
>> --endpoint-url=http://localhost:4566
{
  "S": "512GB SSD"
}
},
"imagen_original_key": {
  "S": "originales/1/imagen-grande.jpg"
},
"miniatura_key": {
  "S": "miniaturas/1/producto-001.jpg"
},
"estado_procesamiento": {
  "S": "ERROR"
}
}
```

E3 — Restauración después del error

```
PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> aws lambda invoke `
>> --function-name lomax-thumbnail `
>> --payload file://lambda/thumbnail/evento-1.json `
>> --cli-binary-format raw-in-base64-out `
>> --endpoint-url=http://localhost:4566 `
>> lambda/thumbnail/respuesta-restauracion.json
{
  "statusCode": 200,
  "ExecutedVersion": "$LATEST"
}
PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> Get-Content .\lambda\thumbnail\respuesta-restauracion.json
{"statusCode": 200, "producto_id": 1, "estado": "LISTA", "imagen_original_key": "originales/1/producto-001.jpg", "miniatura_key": "miniaturas/1/prod
001.jpg", "width": 300, "height": 200}
PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo>
```

E3 — Estado final LISTA después de restauración

```
PS C:\Users\WINDOWS 11\Desktop\lomag-catalogo> aws dynamodb get-item `
>> --table-name ProductoAtributos `
>> --key file://database/clave-producto-1.json `
>> --endpoint-url=http://localhost:4566
{
  "S": "512GB SSD"
}
},
"imagen_original_key": {
  "S": "originales/1/producto-001.jpg"
},
"miniatura_key": {
  "S": "miniaturas/1/producto-001.jpg"
},
"estado_procesamiento": {
  "S": "LISTA"
}
```

Etapa 4. Backend con API y endpoints

- Implementar la API y su Dockerfile.
- El registro base creará un producto pendiente; cargar la imagen completará su publicación.

Endpoint	Operación y verificación esperada
GET /categorias	200: devuelve las categorías almacenadas en RDS.
POST /productos	Recibe código, nombre, descripción, precio, categoria_id y atributos. 201: devuelve producto_id y PENDIENTE. Validar en RDS y DynamoDB.
POST /productos/{id}/imagen	Recibe JPEG o PNG, guarda original e invoca Lambda. 200: PUBLICADO solo tras verificar miniatura y atributos.
POST /productos/{id}/reprocesar	Reintenta la imagen guardada. 200: publica si completa; 409 si no existe original. Repetir no duplica objetos.
GET /productos	200: lista solamente publicados y combina datos principales, atributos y referencia de miniatura.
GET /productos/{id}	200: detalle y estado del producto; 404 si no existe. Permite verificar registros pendientes.
GET /productos/{id}/imagen	200: recupera bytes de la miniatura de S3, con Content-Type correcto; 404 si no está disponible.

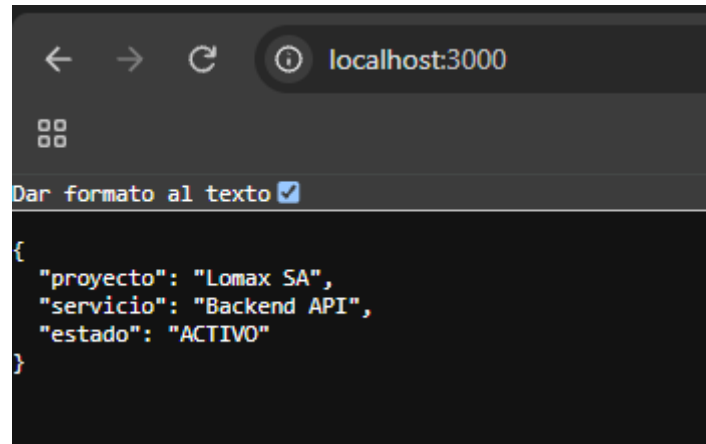
Para entradas inválidas devolver 400, código de producto duplicado 409, archivo mayor a 5 Mb 413 y formato no permitido 415. Si falla un servicio dependiente, informar 502 o 503 y conservar PENDIENTE. Documentar la respuesta que identifica el paso fallido.

Entregable P5: API y endpoints.

Verificación E4: ejecutar todos los endpoints y guardar el reporte.

La prueba exitosa debe verificar directamente RDS, DynamoDB y S3, además de la respuesta HTTP. Comparar el archivo obtenido desde el endpoint de imagen con el descargado.

E4 — Backend API funcionando



E4 — Backend/RDS: conexión a base de datos y producto registrado

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> docker run --rm -it --network floci-red mysql:8.0 mysql -h 172.19.0.2 -P 7001 -u a
-p
mysql> USE lomax;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM productos;
+-----+-----+-----+-----+-----+-----+-----+
| producto_id | codigo | nombre | descripcion | precio | categoria_id | fecha_creacion |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | LOM-001 | Laptop Lenovo IdeaPad | Laptop para uso academico y profesional | 4500.00 | 1 | 2026-09-30 02:31:30 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql>
```

E4 — Backend conectado a RDS MySQL y consultando productos.

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> docker logs lomax-backend

> lomax-backend@1.0.0 start
> node server.js

◇ injected env (0) from .env
◇ injected env (0) from .env
Backend Lomax ejecutándose en http://localhost:3000
Conexión a RDS MySQL: OK
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>
```

E4-02-backend-endpoint

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe http://localhost:3000/
{"proyecto":"Lomax SA","servicio":"Backend API","estado":"ACTIVO"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>
```

E4-03-GET-categorias

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe http://localhost:3000/categorias
[{"categoria_id":1,"nombre":"Laptops","descripcion":"Computadoras portatiles"}, {"categoria_id":2,"nombre":"Monitores","descripcion":"Monitores para computadora"}, {"categoria_id":3,"nombre":"Perifericos","descripcion":"Teclados, mouse y accesorios"}, {"categoria_id":4,"nombre":"Celulares","descripcion":"Telefonos inteligentes"}, {"categoria_id":5,"nombre":"Componentes","descripcion":"Componentes para computadoras"}]
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>
```

E4-04 – POST producto creado – 201

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe -X POST http://localhost:3000/productos `
>> -H "Content-Type: application/json" `
>> --data-binary "@producto-002.json"
{"mensaje":"Producto creado correctamente","producto_id":5,"estado":"PENDIENTE"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>
```

E4-05 – Producto en RDS

```
mysql> SELECT
->     producto_id,
->     codigo,
->     nombre,
->     precio,
->     categoria_id,
->     estado
-> FROM productos
-> WHERE producto_id = 5;
+-----+-----+-----+-----+-----+-----+
| producto_id | codigo | nombre      | precio | categoria_id | estado |
+-----+-----+-----+-----+-----+-----+
|          5 | LOM-002 | Monitor LG 24 | 1200.00 |          2 | PENDIENTE |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

E4-06 – Atributos del producto en DynamoDB

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\database> aws dynamodb get-item `
>> --table-name ProductoAtributos `
>> --key file://clave-producto-5.json `
>> --endpoint-url=http://localhost:4566
{"producto_id": {
  "N": "5"
},
"atributos": {
  "M": {
    "resolucion": {
      "S": "1920x1080"
    },
    "tamano": {
      "S": "24 pulgadas"
    }
  }
}
```

E4-07 – Código de producto duplicado

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe -X POST http://localhost:3000/productos `
>> -H "Content-Type: application/json" `
>> --data-binary "@C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend\producto-duplicado.json"
{"error":"El código del producto ya existe"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>
```

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe -i -X POST http://localhost:3000/productos `
>> -H "Content-Type: application/json" `
>> --data-binary "@C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend\producto-duplicado.json"
HTTP/1.1 409 Conflict
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 45
ETag: W/"2d-a52tF6ND9MvzB8DYLCxbG4ckQwc"
Date: Wed, 30 Sep 2026 07:54:31 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"El código del producto ya existe"}

```

E4-08 – Precio negativo rechazado – HTTP 400

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe -i -X POST http://localhost:3000/productos `
>> -H "Content-Type: application/json" `
>> --data-binary "@C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend\producto-precio-negativo.json"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 43
ETag: W/"2b-qCWrCuPQKhGrFBKpXTs1KLIUXS4"
Date: Wed, 30 Sep 2026 07:55:38 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"El precio no puede ser negativo"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>

```

E4-09-Categoria-inexistente-400

```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> curl.exe -i -X POST http://localhost:3000/productos `
>> -H "Content-Type: application/json" `
>> --data-binary "@C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend\producto-categoria-inexistente.json"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 35
ETag: W/"23-5+n4t0BHNr3x5uukSXJSy2ITTV"
Date: Wed, 30 Sep 2026 07:58:44 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"La categoría no existe"}
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>

```

E4-10-Backend-actualizado


```

PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> docker run -d `
>> --name lomax-backend `
>> --network floci-red `
>> -p 3000:3000 `
>> --env-file .env `
>> -e AWS_ENDPOINT_URL=http://floci:4566 `
>> -e AWS_ACCESS_KEY_ID=test `
>> -e AWS_SECRET_ACCESS_KEY=test `
>> -e AWS_REGION=us-east-1 `
>> lomax-backend:latest
d53ffa06ffa0cf50aec97665e0339f1a53a7b0110d21269b0fa21e384ccf10d9
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend> docker logs lomax-backend

> lomax-backend@1.0.0 start
> node server.js

◇ injected env (0) from .env
◇ injected env (0) from .env
Backend Lomax ejecutándose en http://localhost:3000
Conexión a RDS MySQL: OK
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\backend>

```

Etaapa 5. Frontend con dashboard del catálogo

Construir las tres vistas: registrar producto, catálogo y detalle.

- El formulario incluirá atributos variables y fotografía; mostrará validaciones, estado de registro, errores y opción de reintento.
- El catálogo mostrará miniatura, nombre, precio y categoría.
- El detalle incluirá descripción y atributos.

Integrar frontend, backend y proxy con Docker Compose, redes y variables. Mostrar datos provenientes de los servicios y conservar la información al actualizar el navegador. No se requieren indicadores de ventas ni gestión de inventario.

Entregable P6: Frontend.

Verificación E5: registrar un producto nuevo desde el formulario, observar su publicación y mostrar su detalle. Entregar capturas con producto_id, más solicitudes y respuestas HTTP. Confirmar el registro en RDS y DynamoDB y descargar su miniatura desde S3.

Probar código duplicado y archivo inválido: la interfaz debe informar el error y el producto incompleto no debe aparecer en el catálogo. Demostrar los 20 productos publicados en el catálogo y en las consultas de datos.

E5-1 Frontend

LOMAX SA

Catálogo de productos

[+ Registrar producto](#)

MONITORES

Monitor LG 24

Código: LOM-002

Monitor Full HD para
computadora**Bs. 1200.00**[Ver detalle](#)[← Volver al catálogo](#)

PORTÁILES

Portátil HP ProBook

Código: LOM-009

Bs. 4800.00

Laptop empresarial para trabajo y estudio.

Características**RAM : 16 GB Almacenamiento : 512 GB****Conexión SSD : USB-C**

Estado: PUBLICADO

Formulario de registro de producto:

- Código ***: Ej. LOM-003
- Nombre ***: Ej. Portátil Lenovo
- Descripción**: Descripción del producto
- Precio ***: 4500.00
- Categoría ***: Seleccionar categoría
- Atributos variables ***: RAM: 16 GB, almacenamiento: 512 GB SSD, conexión: USB-C. Ejemplo: RAM: 16 GB, almacenamiento: SSD de 512 GB
- Fotografía ***: Seleccionar archivo. Ningún archivo seleccionado. Formatos permitidos: JPEG o PNG. Tamaño máximo: 5 MB.
- Botón**: Registrar y publicar producto

Etapa 6. Publicación de imágenes en ECR

Crear repositorios ECR en FLOCI para frontend y backend. Construir, etiquetar y publicar las imágenes propias con una versión vinculada al commit de entrega. Documentar autenticación, dirección del registro y conectividad desde Docker y Kubernetes.

Entregable P7: Imágenes en ECR.

Verificación E6: adjuntar las salidas exitosas de docker push y consultar ecr describe-images o list-images en FLOCI.

Descargar cada imagen mediante docker pull usando su referencia remota, preferentemente desde un entorno sin copia previa.

Ejecutar las imágenes descargadas y consultar la API y el frontend para comprobar que contienen la aplicación entregada. Las imágenes se consumirán desde ECR en la etapa siguiente.

Etapa 7. Despliegue y validación en EKS

Crear el clúster EKS mediante FLOCI. Configurar acceso a las imágenes de ECR y a RDS, DynamoDB, S3 y Lambda desde los Pods. El proxy seguirá siendo la entrada de usuarios.

RDS y DynamoDB proporcionan persistencia fuera de los Pods de aplicación. Mantener el almacenamiento del entorno FLOCI configurado en las etapas anteriores.

Entregable P8: EKS.

Verificación E7: Mostrar los pods. Acreditar que EKS ejecuta las imágenes publicadas en ECR.

- Escalar backend de 1 a 3 réplicas. Mostrar las tres Ready y solicitudes atendidas por cada Pod mediante logs o un identificador de instancia en las respuestas.
- Eliminar un Pod y registrar su UID antes y el UID del reemplazo después. Mostrar recuperación del número de réplicas y funcionamiento del catálogo.
- Registrar un producto nuevo desde la aplicación en EKS, verificar datos en RDS y DynamoDB, descargar su miniatura desde S3 y comprobar su aparición en el catálogo. Recrear Pods y repetir consultas: el mismo producto debe permanecer disponible.

Portafolio de evidencias técnicas

Entregable P9: repositorio con README, scripts, configuración de ejemplo, código, diagramas. Organizar evidencias E1 a E7 por etapa.

E7-1. Ejecución de pull localhost

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\frontend> docker tag lomax-frontend:latest localhost:5100/lomax-frontend:v2
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\frontend> docker push localhost:5100/lomax-frontend:v2
The push refers to repository [localhost:5100/lomax-frontend]
d9aae54b5831: Layer already exists
e2de96513ba9: Layer already exists
2e2b6dd99e8e: Pushed
44136fa355b3: Already exists
6c53d0b2a666: Layer already exists
e76228b47809: Layer already exists
004b2032f5bd: Pushed
9a9a644fdd6a: Layer already exists
745dfb2690dd: Layer already exists
64c8194480fe: Layer already exists
e72112c14215: Layer already exists
v2: digest: sha256:165825949f66791886bd9cc5a9297a8e50c4cafde1342b84b63cae5549cf9820 size: 856
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\frontend>
```

E7-2 Verificación de los Pods

```
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\kubernetes> $env:AWS_ACCESS_KEY_ID="AKIA1PHOVM9GKPER1G6N"
>> $env:AWS_SECRET_ACCESS_KEY="YvTHN54YRIY/aHnFg34p1B1ZFmeKthqaGjBRP08k"
>> $env:AWS_DEFAULT_REGION="us-east-1"
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\kubernetes> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lomax-backend-74cb4dbf6c-5tvvz      1/1     Running   0           13m
lomax-frontend-748445d6c8-rkgdv     1/1     Running   0           8m3s
lomax-proxy-7cbb69d6f-z9zsc         1/1     Running   0           4m7s
PS C:\Users\WINDOWS 11\Desktop\lomax-catalogo\kubernetes> kubectl port-forward svc/lomax-proxy 8082:80
Forwarding from 127.0.0.1:8082 -> 80
Forwarding from [::1]:8082 -> 80
Handling connection for 8082
Handling connection for 8082
Handling connection for 8082
```

DEBE SER

Organización y trabajo colaborativo

Cada integrante debe aportar trabajo verificable y comprender el flujo completo. Asignar roles y tareas por etapa, mantener su estado en un tablero y relacionarlas con los entregables.

- Identificar a cada miembro y su cuenta Github. Presentar commits sustantivos de código, pruebas, scripts, diseño o documentación, vinculados con las tareas; la cantidad de commits por sí sola no demuestra participación.

Entregable P10: Repositorio de Github.

Entregar enlace de repositorio en Github.

Verificación: se seleccionará aportes de cada integrante, revisará el cambio o archivo y solicitará su explicación y demostración. Cualquier integrante podrá ser consultado sobre otras partes de la solución.

ENLACE: <https://github.com/brendachallco7-spec/lomax-catalogo>

DEFENSA DEL PROYECTO

La defensa combinará preguntas del SABER CONOCER con retos sobre los entregables de las siete etapas. Se evaluará comprensión individual y capacidad de verificar el funcionamiento del sistema.

- Explicar el diagrama de arquitectura.
- Explicar el recorrido de un producto y por qué un registro incompleto permanece pendiente.
- Demostrar rechazo de un código duplicado, consultar atributos variables y verificar persistencia tras reinicio.
- Localizar y descargar la miniatura desde S3, comprobar sus dimensiones y repetir la invocación de Lambda sin duplicados.
- Ejecutar endpoints con datos válidos e inválidos, interpretar respuestas y completar un registro pendiente sin crear otro producto.
- Registrar y consultar un producto desde el dashboard y demostrar que sus datos provienen de los servicios.
- Mostrar push, digest remoto, pull y ejecución de una imagen recuperada desde ECR.
- Relacionar las imágenes de los Pods con ECR, escalar a tres réplicas y demostrar reemplazo de un Pod sin pérdida de productos.

Las respuestas conceptuales deberán relacionarse con las decisiones implementadas.

CRITERIOS DE EVALUACIÓN

Criterio	Descripción	%
Fundamentación	P1: comprende los conceptos y justifica el uso de los seis servicios en el catálogo.	5%
Arquitectura	P2 / E1: diagrama con íconos oficiales, flujos, estados y correspondencia final con la implementación.	5%
Persistencia	P3 / E2: modelo relacional, atributos variables, restricciones y datos conservados tras reinicio.	5%
S3 y Lambda	P4 / E3: miniatura descargada y dimensionada, referencia coincidente y reintentos sin duplicados.	5%
Backend	P5 / E4: endpoints, validación, idempotencia y pruebas verificables contra los servicios.	5%
Frontend	P6 / E5: registro, catálogo y detalle con datos reales; estados y errores visibles en Compose.	5%
ECR	P7 / E6: push y pull reales, coincidencia de digests y ejecución de imágenes descargadas.	5%
EKS	P8 / E7: imágenes de ECR, aplicación integrada, tres réplicas, recuperación y persistencia.	5%
Defensa técnica	P2/E1 diseño; P3/E2 persistencia; P4/E3 S3 y Lambda; P5/E4 backend; P6/E5 frontend; P7/E6 ECR; P8/E7 EKS. Explica y demuestra cada reto.	50%
Colaboración	P10: aportes atribuibles de todos, vinculados a tareas y entregables, con revisión y explicación individual.	10%
Total		100%